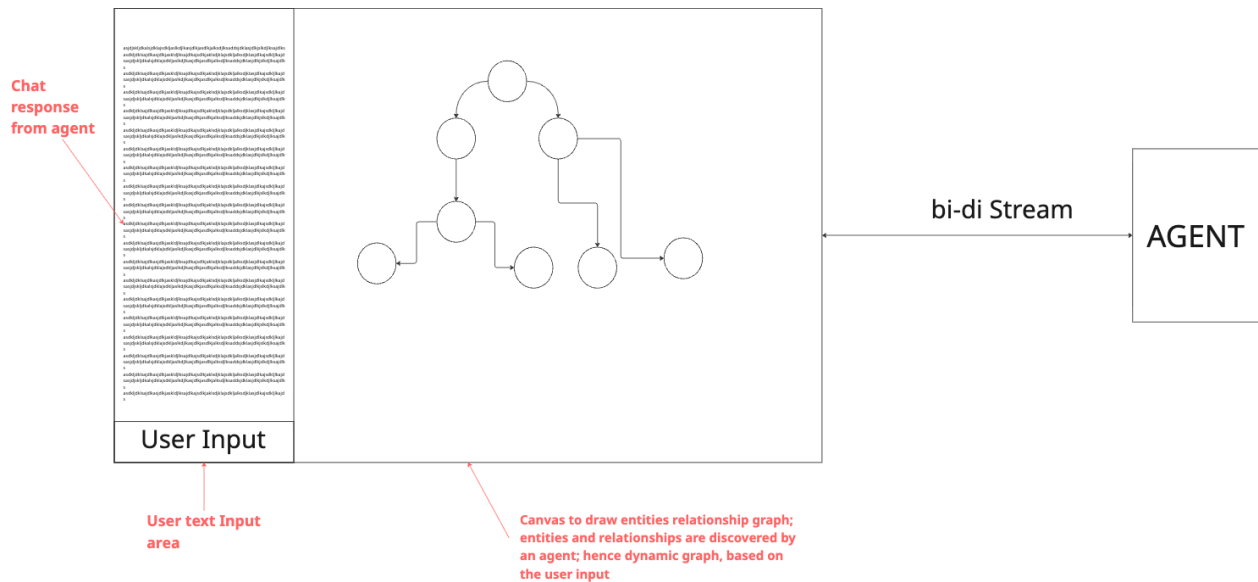


Frontend Architecture Design Exercise



Design the frontend architecture for the application described above. Produce a document that discusses your design coherently – no code artifacts are expected, though you may use short snippets to support your reasoning.

Address the following:

1. The Challenge — one connection, many surfaces

The browser must show, in real time: agent thinking (spinner), streaming text, tool execution cards, map updates, and graph nodes. Discuss:

- How should the frontend talk to the agent backend — SSE, WebSocket, or REST?
- Why?
- One stream for everything, or one per UI segment?

2. Wire Format

What should the messages on the wire look like? For example, an SSE stream of line-by-line JSON events:

```
data: {type: "TOOL_CALL_START", toolCallId, toolCallName, parentMessageId}
data: {type: "TOOL_CALL_ARGS", toolCallId, delta: "{\"signals\":[...]}"
data: {type: "STATE_SNAPSHOT", snapshot: {signals, viewport, entity_canvas, ...}}
data: {type: "TOOL_CALL_END", toolCallId}
```

3. State Coherence

How do frontend and agent stay consistent — for both chat and canvas — given that:

- The agent emits a state snapshot after each tool completes, not before
- The frontend must apply state updates in order (FIFO)
- Map shapes/signals persist across tool calls unless explicitly cleared
- User-drawn shapes (frontend-owned state) must survive agent updates

4. Session Continuity

How does a user reload or reconnect and land exactly where they left off?

5. Bidirectional Binding

- How does the frontend send its state (viewport, shapes, tags) to the agent?
- How does the agent mutate the canvas state on the frontend?
- Overall: how is the state kept in sync in both directions?

6. Nondeterministic UI control.

The agent decides at runtime which surfaces to touch.

- How do you keep a human designer in control of UX quality when the "caller" of your components is an LLM?

Key discussion points we are looking for in your document:

Stream processing, State management, resilience, replay-&-resume, concurrency, architecture judgements, testing, Non-deterministic UI control.